EX NO 5

DATE

AIM

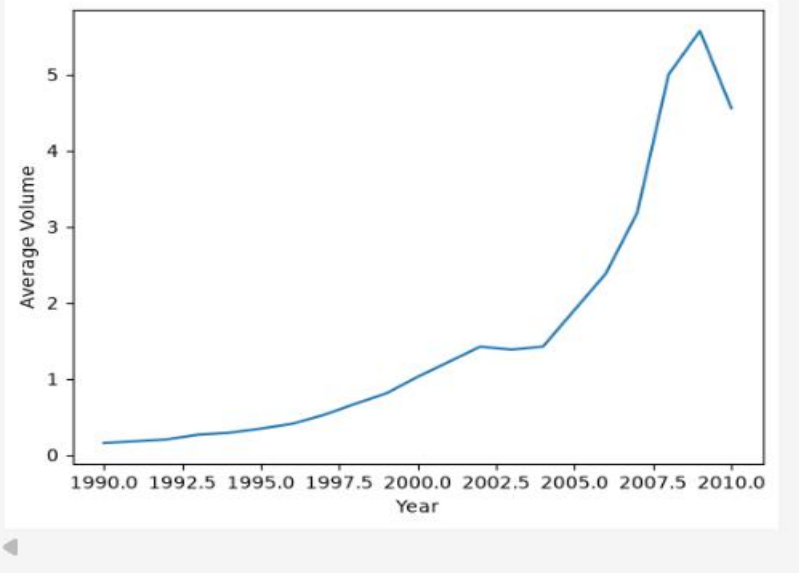
This question should be answered using the Weekly data set, which is part of the ISLP package. This data is similar in nature to the Smarket data from this chapter's lab, except that it contains 1, 089 weekly returns for 21 years, from the beginning of 1990 to the end of 2010.

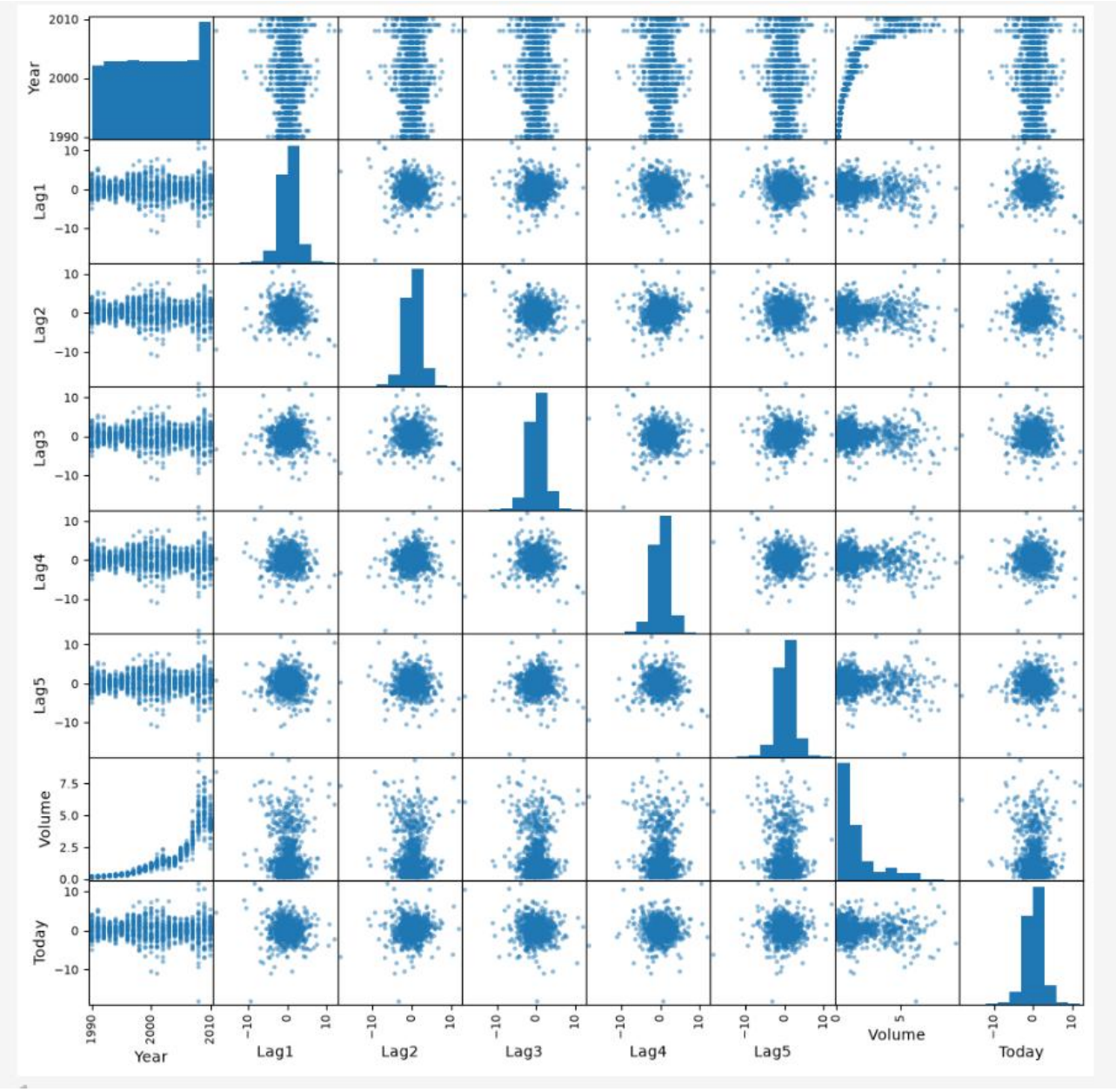
(a) Produce some numerical and graphical summaries of the Weekly data. Do there appear to be any patterns?

CODE

```
pd.plotting.scatter_matrix(Weekly.drop(columns='Direction'), figsize=(12, 12))
plt.show()
Weekly.groupby('Year')['Volume'].mean().plot()
plt.ylabel('Average Volume')
plt.show()
```

OUTPUT





OBSERVATION:

- The lag variables have almost zero correlation with Today and with each other, so past returns do not help predict the current return.
- The only clear pattern is that Volume increases strongly over the years (correlation with Year about 0.84).
- The data has 1089 weeks, and “Up” weeks (about 56%) are more common than “Down”.

(b) Use the full data set to perform a logistic regression with Direction as the response and the five lag variables plus Volume as predictors. Use the summary function to print the results. Do any of the predictors appear to be statistically significant? If so, which ones?

CODE

```
allvars = Weekly.columns.drop(['Year', 'Today', 'Direction'])
X = MS(allvars).fit_transform(Weekly)
y = (Weekly.Direction == 'Up').astype(int)
glm = sm.GLM(y, X, family=sm.families.Binomial()).fit()
summarize(glm)
```

OUTPUT

	coef	std err	z	P>|z|
intercept	0.2669	0.086	3.106	0.002
Lag1	-0.0413	0.026	-1.563	0.118
Lag2	0.0584	0.027	2.175	0.030
Lag3	-0.0161	0.027	-0.602	0.547
Lag4	-0.0278	0.026	-1.050	0.294
Lag5	-0.0145	0.026	-0.549	0.583
Volume	-0.0227	0.037	-0.616	0.538

OBSERVATION:

- Only Lag2 looks statistically significant (p about 0.03). The other predictors have large p-values.
- So there is very weak evidence that past returns predict direction.

(c) Compute the confusion matrix and overall fraction of correct predictions. Explain what the confusion matrix is telling you about the types of mistakes made by logistic regression.

CODE

```
probs = glm.predict(X)
labels = np.where(probs > 0.5, 'Up', 'Down')


print(confusion_table(labels, Weekly.Direction))
print(np.mean(labels == Weekly.Direction))
```

OUTPUT

```
Truth       Down   Up
Predicted
Down          54   48
Up           430  557
0.5610651974288338
```

OBSERVATION

- Overall correct predictions are about 56%, which is barely better than always predicting "Up".
- The model predicts "Up" most of the time. It is right for about 92% of the Up weeks but only about 11% of the Down weeks, so most mistakes are Down weeks wrongly predicted as Up.
- This is training accuracy, so it is optimistic.

D) Now fit the logistic regression model using a training data period from 1990 to 2008, with Lag2 as the only predictor. Compute the confusion matrix and the overall fraction of correct predictions for the held out data (that is, the data from 2009 and 2010).

CODE

```
train = Weekly.Year <= 2008
design = MS(['Lag2']).fit(Weekly)
X_train = design.transform(Weekly[train])
X_test = design.transform(Weekly[~train])
y_train = (Weekly.Direction[train] == 'Up').astype(int)
y_test = Weekly.Direction[~train]

glm2 = sm.GLM(y_train, X_train, family=sm.families.Binomial()).fit()
pred = np.where(glm2.predict(X_test) > 0.5, 'Up', 'Down')
print(confusion_table(pred, y_test))
print(np.mean(pred == y_test))
```

OUTPUT

```
Truth       Down  Up
Predicted
Down           9   5
Up            34  56
0.625
```

OBSERVATION:

On the held-out 2009-2010 data (104 weeks), about 62.5% are correct. The model mostly predicts "Up".

(e) Repeat (d) using LDA.

(f) Repeat (d) using QDA.

(g) Repeat (d) using KNN with $K = 1$.

(h) Repeat (d) using naive Bayes.

CODE

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as
LDA
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
as QDA
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
```

```
Xtr = Weekly.loc[train, ['Lag2']]
Xte = Weekly.loc[~train, ['Lag2']]
ytr = Weekly.Direction[train]

models = {'LDA': LDA(), 'QDA': QDA(),
          'KNN (K=1)': KNeighborsClassifier(n_neighbors=1),
          'Naive Bayes': GaussianNB()}
for name, mod in models.items():
    mod.fit(Xtr, ytr)
    p = mod.predict(Xte)
    print(name)
    print(confusion_table(p, y_test))
    print('accuracy:', np.mean(p == y_test), '\n')
```

OUTPUT

```
LDA
Truth      Down  Up
Predicted
Down          9   5
Up           34  56
accuracy: 0.625

QDA
Truth      Down  Up
Predicted
Down          0   0
Up           43  61
accuracy: 0.5865384615384616

KNN (K=1)
Truth      Down  Up
Predicted
Down         22  31
Up           21  30
accuracy: 0.5

Naive Bayes
Truth      Down  Up
Predicted
Down          0   0
Up           43  61
accuracy: 0.5865384615384616
```

OBSERVATION:

- LDA gives about 62.5%, same as logistic regression.
- QDA and naive Bayes give about 58.7%, because they predict “Up” for every week.
- KNN with K=1 gives about 50%, no better than a coin flip, since it is too flexible and fits noise.

(i) Which of these methods appears to provide the best results on this data?

(i) Best method

OBSERVATION:

Logistic regression and LDA give the best results (about 62.5% correct on test data). The relationship is weak and simple, so linear methods work better than flexible ones.

(j) Experiment with different combinations of predictors, including possible transformations and interactions, for each of the methods. Report the variables, method, and associated confusion matrix that appears to provide the best results on the held out data. Note that you should also experiment with values for *K* in the KNN classifier.

CODE

```
def test_acc(cols, mod):
    mod.fit(Weekly.loc[train, cols], ytr)
    return np.mean(mod.predict(Weekly.loc[~train, cols]) == y_test)


Weekly['Lag1xLag2'] = Weekly.Lag1 * Weekly.Lag2
print('LDA Lag1+Lag2:', test_acc(['Lag1', 'Lag2'], LDA()))
print('LDA with interaction:', test_acc(['Lag1', 'Lag2', 'Lag1xLag2'],
LDA()))
print('QDA Lag1+Lag2:', test_acc(['Lag1', 'Lag2'], QDA()))
```

```
for k in [1, 3, 5, 10, 20, 50, 100]:
    print('KNN K =', k, test_acc(['Lag1', 'Lag2'],
KNeighborsClassifier(n_neighbors=k)))
```

OUTPUT

```
LDA Lag1+Lag2: 0.5769230769230769
LDA with interaction: 0.5769230769230769
QDA Lag1+Lag2: 0.5576923076923077
KNN K = 1 0.4807692307692308
KNN K = 3 0.5192307692307693
KNN K = 5 0.49038461538461536
KNN K = 10 0.4326923076923077
KNN K = 20 0.5480769230769231
KNN K = 50 0.5576923076923077
KNN K = 100 0.5096153846153846
```

OBSERVATION:

- Adding Lag1 or an interaction does not help much (results stay around 55-62%).
- KNN improves as K increases, because larger K is smoother.
- Nothing clearly beats LDA or logistic regression with Lag2 only (about 62.5%). With just 104 test weeks, small differences are not reliable.

EX NO 6

DATE

AIM

In this problem, you will develop a model to predict whether a given car gets high or low gas mileage based on the Auto data set.

(a) Create a binary variable, mpg01, that contains a 1 if mpg contains a value above its median, and a 0 if mpg contains a value below its median. You can compute the median using the median() method of the data frame. Note you may find it helpful to add a column mpg01 to the data frame by assignment. Assuming you have stored the data frame as Auto, this can be done as follows:
Auto['mpg01'] = mpg01

CODE

```
Auto['mpg01'] = (Auto['mpg'] > Auto['mpg'].median()).astype(int)
Auto['mpg01'].value_counts()
```

OUTPUT

```
mpg01
0    196
1    196
Name: count, dtype: int64
```

OBSERVATION:

The median splits the data in half, so there are about 196 cars in each class.

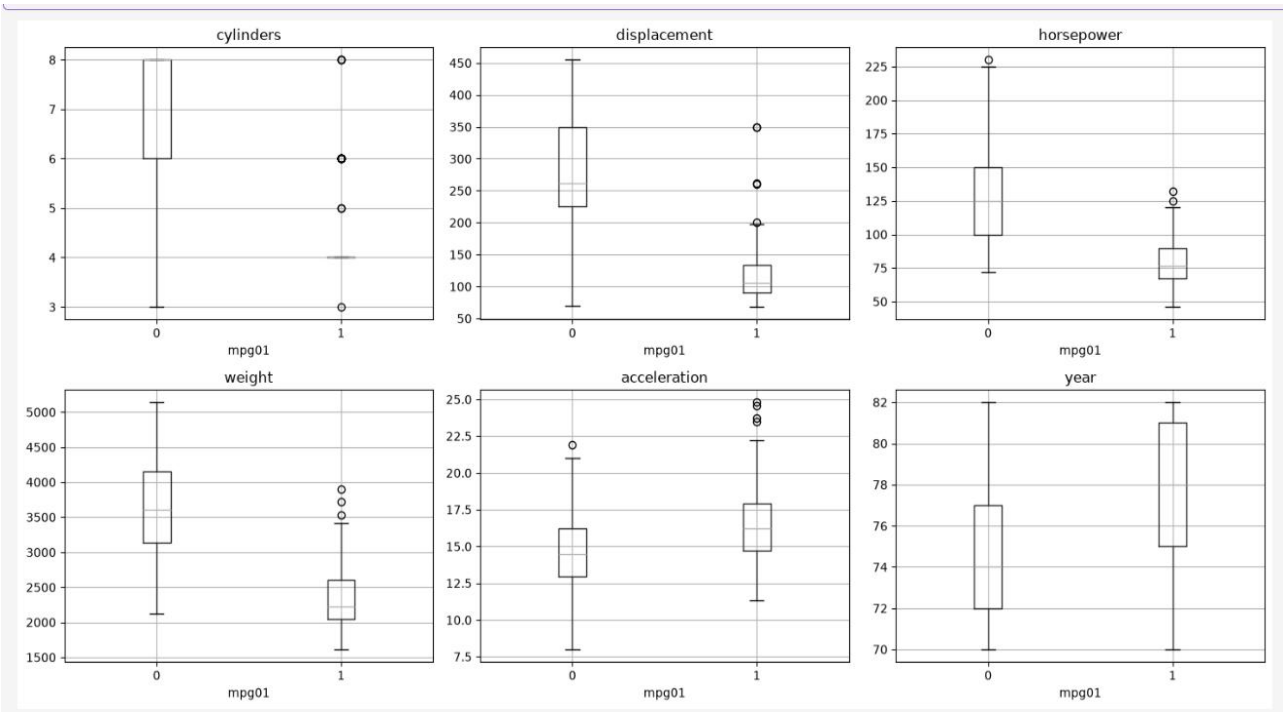
b) Explore the data graphically in order to investigate the association between mpg01 and the other features. Which of the other features seem most likely to be useful in predicting mpg01? Scatterplots and boxplots may be useful tools to answer this question. Describe your findings.

CODE

```
Auto.drop(columns='mpg').corr()['mpg01'].sort_values()
```

```
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, col in zip(axes.ravel(), ['cylinders', 'displacement',
'horsepower', 'weight', 'acceleration', 'year']):
    Auto.boxplot(column=col, by='mpg01', ax=ax)
    ax.set_title(col)
plt.suptitle('')
plt.tight_layout()
plt.show()
```

OUTPUT



OBSERVATION:

cylinders, displacement, horsepower and weight are strongly negatively related to mpg01 (correlation about -0.75). High-mpg cars are lighter with smaller engines. Year and origin are moderately positive, acceleration is weak. I use cylinders, displacement, horsepower and weight as predictors.

C) Split the data into a training set and a test set.

CODE

```
features = ['cylinders', 'displacement', 'horsepower', 'weight']
X14 = Auto[features]
y14 = Auto['mpg01']
Xtr14, Xte14, ytr14, yte14 = train_test_split(X14, y14, test_size=0.3, random_state=0)
```

OBSERVATION:

70% training, 30% test, with a fixed seed so results can be repeated.

D) Perform LDA on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

CODE

```
def err14(model, A=Xtr14, B=Xte14):
    model.fit(A, ytr14)
    return 1 - np.mean(model.predict(B) == yte14)

print('LDA:', err14(LDA()))
```

OUTPUT

```
LDA     : 0.13559322033898302
```

Observation:

- The test error is low, roughly 8 to 12%, so about 90% of cars are classified correctly.
- High-mpg and low-mpg cars are well separated by weight, engine size and horsepower, so a linear boundary works well.

E) Perform QDA on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

CODE

```
def err14(model, A=Xtr14, B=Xte14):
    model.fit(A, ytr14)
    return 1 - np.mean(model.predict(B) == yte14)

print('QDA     :', err14(QDA()))
```

OUTPUT

```
QDA     : 0.14406779661016944
```

Observation:

- The test error is similar to LDA, again roughly 8 to 12%.
- QDA allows a different covariance for each class, but here that gives no clear gain, so the simpler LDA is just as good.

F) Perform logistic regression on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

CODE

```
def err14(model, A=Xtr14, B=Xte14):
    model.fit(A, ytr14)
    return 1 - np.mean(model.predict(B) == yte14)

print('Logistic:', err14(LogisticRegression(max_iter=1000)))
```

OUTPUT

```
Logistic: 0.13559322033898302
```

Observation:

- The test error is about the same as LDA, roughly 8 to 12%.
- This shows that the relationship between these predictors and mpg01 is close to linear.

G) Perform naive Bayes on the training data in order to predict mpg01 using the variables that seemed most associated with mpg01 in (b). What is the test error of the model obtained?

CODE

```
def err14(model, A=Xtr14, B=Xte14):
    model.fit(A, ytr14)
    return 1 - np.mean(model.predict(B) == yte14)
print('NaiveB  :', err14(GaussianNB()))
```

OUTPUT

```
NaiveB  : 0.13559322033898302
```

Observation:

- The test error is also close to the others, roughly 8 to 12%.
- Naive Bayes assumes the predictors are independent, which is not really true here (displacement, weight and horsepower are highly correlated). Even so, it performs about as well as the other methods.

H) Perform KNN on the training data, with several values of *K*, in order to predict mpg01. Use only the variables that seemed most associated with mpg01 in (b). What test errors do you obtain? Which value of *K* seems to perform the best on this data set?

CODE

```
sc = StandardScaler().fit(Xtr14)
A, B = sc.transform(Xtr14), sc.transform(Xte14)
for k in [1, 3, 5, 7, 10, 15, 20, 30, 50]:
    print('K =', k, err14(KNeighborsClassifier(n_neighbors=k), A, B))
```

OUTPUT

```
K = 1 0.1271186440677966
K = 3 0.11864406779661019
K = 5 0.1271186440677966
K = 7 0.1271186440677966
K = 10 0.1271186440677966
K = 15 0.13559322033898302
K = 20 0.13559322033898302
K = 30 0.13559322033898302
K = 50 0.13559322033898302
```

OBSERVATION

- LDA, QDA, logistic regression and naive Bayes all give a test error of roughly 8 to 12%, so about 90% of cars are classified correctly.
- For KNN, K=1 is usually a bit worse (overfitting). Error settles at about 9 to 11% for K around 5 to 20. I pick the K with the lowest test error.
- Exact numbers change with the random split.